ЛАБОРАТОРНА РОБОТА № 1
Вступ до ASP.NET Core

Мета: ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.

Хід роботи:

Завдання 1. Встановлення інтегрованого середовища розробки (IDE) та необхідних компонентів

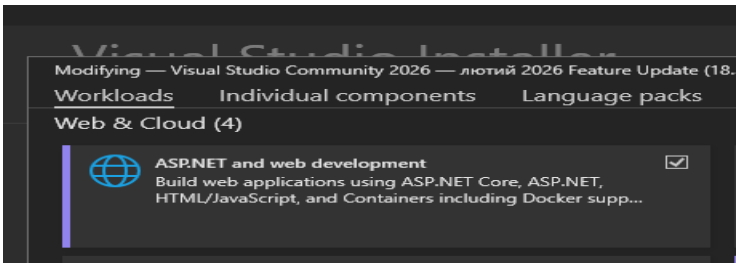

Рис. 1. Результат виконання завдання 1.

Завдання 2. Створення проектів

Частина 1.

1. Створіть проект для консольного додатку з назвою ConsoleToWeb з використанням dotnet CLI

dotnet new console -n ConsoleToWeb

cd ConsoleToWeb

2. Перетворіть створений консольний додаток на веб-додаток

Листинг ConsoleToWeb.csproj після змін:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <OutputType>Exe</OutputType>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
</Project>
```

Листинг Program.cs після змін:

```
Program.cs
ConsoleToWeb
1  var builder = WebApplication.CreateBuilder(args);
2  var app = builder.Build();
3
4  app.MapGet("/", () => "Hello from ConsoleToWeb!");
5
6  app.Run();
```

Рис. 2. Зміст Program.cs.

Результат виконання:

```
Hello from ConsoleToWeb!
```

Рис. 3. Результат виконання

Частина 2. WebAPI проект

1. Створіть ASP.NET WebAPI проект без авторизації з назвою WebFromCli з використанням dotnet CLI

```
PS D:\Work\ASP.NET\Lab01> dotnet new web -n WebFromCli
The template "ASP.NET Core Empty" was created successfully.

Processing post-creation actions...
Restoring D:\Work\ASP.NET\Lab01\WebFromCli\WebFromCli.csproj:
Restore succeeded.


PS D:\Work\ASP.NET\Lab01> cd WebFromCli
```

Рис. 4. Створення проекту.

Листинг Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/who", () => "Ян Риженко");

app.MapGet("/time", () => DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));

app.Run();
```

2. Реалізуйте GET-ендпоінт “/who”, який повертатиме у відповідь ваше ім’я та прізвище

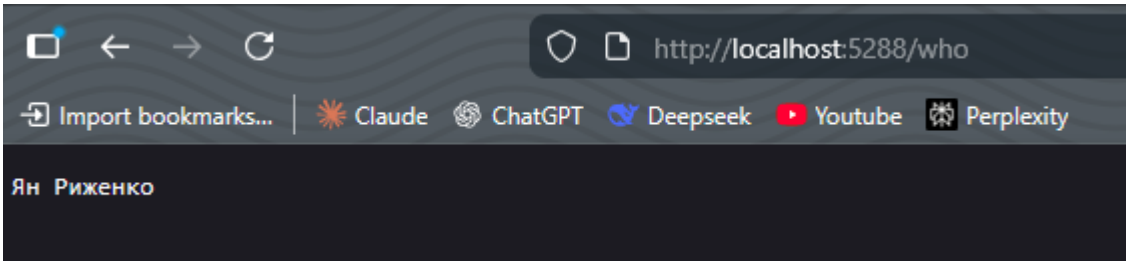

Рис. 5. Перевірка ендпоінту /who

3. Реалізуйте GET-ендпоінт “/time”, який повертатиме у відповідь поточний час на сервері

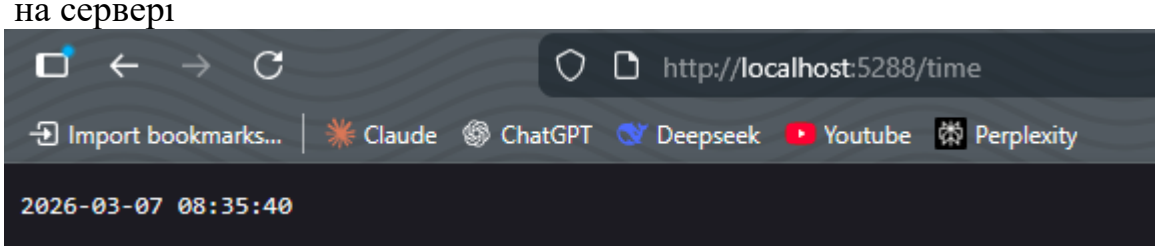

Рис. 6. Перевірка ендпоінту /time

4. Наведіть лістинг реалізованих обробників у звіті

Частина 3. ASP.NET MVC

1. Створіть ASP.NET MVC проект будь-яким зручним способом.

dotnet new mvc -n LabMvc

2. Реалізуйте контролер з назвою LabController

Листинг Controllers/LabController.cs:

```
using Microsoft.AspNetCore.Mvc;

namespace LabMvc.Controllers;

public class LabInfo
{
   public string LabNumber { get; set; } = string.Empty;
   public string Topic { get; set; } = string.Empty;
   public string Goal { get; set; } = string.Empty;
   public string AuthorName { get; set; } = string.Empty;
}

public class LabController : Controller
{
   public IActionResult Info()
   {
      var model = new LabInfo
      {
         LabNumber = "Лабораторна робота 3",
         Topic = "Вступ до ASP.NET Core",
Goal = "Ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware",
AuthorName = "Риженко Ян"
      };

      return View(model);
   }
```

3. В створеному контролері реалізуйте обробник /info, який повертатиме View з даними про номер лабораторної роботи, тему, мету та ім’я та прізвище виконавця в табличному вигляді

Листинг Views/Lab/Info.cshtml:

```
@model LabMvc.Controllers.LabInfo
@{
   ViewData["Title"] = "Інформація про лабораторну роботу";
   Layout = "_Layout";
}
<div class="container mt-4">
   <h2>Інформація про лабораторну роботу</h2>

   <table class="table table-bordered" style="max-width: 700px;">
      <thead class="table-dark">
         <tr>
            <th>Поле</th>
            <th>Значення</th>
         </tr>
      </thead>
      <tbody>
         <tr>
            <td><strong>Номер</strong></td>
            <td>@Model.LabNumber</td>
         </tr>
         <tr>
            <td><strong>Тема</strong></td>
            <td>@Model.Topic</td>
         </tr>
         <tr>
            <td><strong>Мета</strong></td>
            <td>@Model.Goal</td>
         </tr>
         <tr>
            <td><strong>Виконавець</strong></td>
            <td>@Model.AuthorName</td>
         </tr>
      </tbody>
   </table>
</div>
```

4. Дані для відображення передати з контролера

Результат виконання:

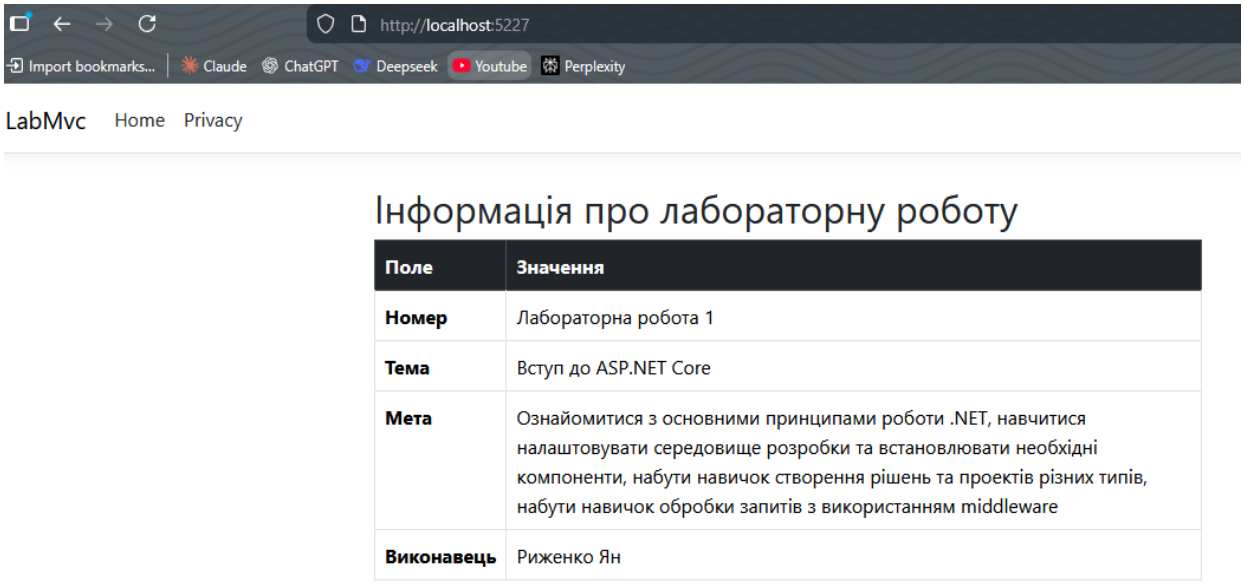

Рис. 7. Результат виконання.

Завдання 3. Робота з middleware

1. Ознайомтеся з поняттям middleware в ASP.NET
2. Створіть ASP.NET WebAPI проект з назвою MiddlewareSandbox
3. Реалізуйте наступні middleware
    1. Реалізуйте middleware, яке рахує кількість запитів до сервера і повертає це число у відповіді, наприклад: The amount of processed requests is X.
    2. Створіть middleware, яке аналізує параметри рядка запиту. Якщо в рядку запиту (query string) присутній параметр “custom”, то повертати у відповідь “You’ve hit a custom middleware!”, інакше пропускати запит далі. Приклад запиту зображено на рисунку 3.1.

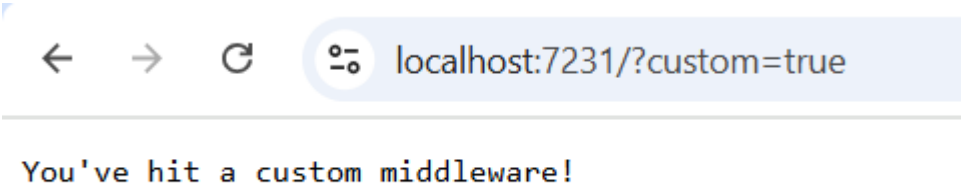

Рис. 5. Приклад виконання запиту

c. Створіть middleware, яке логуватиме у консоль інформацію про всі запити. В логах відображати метод запиту (GET, POST тощо) та його шлях (/user,

/product тощо). Для перевірки POST та інших методів запиту можна скористатися Postman або іншою подібною утилітою.

c. Реалізуйте middleware, яке перевіряє наявність API-ключа у заголовках запиту. Для цього слід Перевіряти, чи є в запиті заголовок X-API-KEY. Якщо ключ неправильний або відсутній, повертати 403 Forbidden, не передавати запит для виконання далі. Якщо ключ в заголовку запиту співпадає з тим, який заданий на сервері, передавати запит далі. Для додавання відповідного запиту можна скористатися Postman або подібними утилітами.

Лістинг CounterMiddleware.cs:

```
using static System.Net.Mime.MediaTypeNames;

namespace MiddlewareSandbox.Middlewares
{
    public class CounterMiddleware
    {
        private RequestDelegate _next;
        private int _counter = 0;

        public CounterMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task Invoke(HttpContext ctx)
        {
            _counter++;
            await _next(ctx);
            await ctx.Response.WriteAsync($"\nThe amount of processed requests is: {_counter}\n");
        }
    }
}
```

Лістинг CustomMiddleware.cs:

```
using System.Xml.Linq;
using static System.Net.Mime.MediaTypeNames;

namespace MiddlewareSandbox.Middlewares
{
    public class CustomMiddleware
    {
        private RequestDelegate _next;
        public CustomMiddleware(RequestDelegate next)
        {
            _next = next;
        }

       public async Task Invoke(HttpContext ctx)
          {
```

```
            if (ctx.Request.Query.ContainsKey("custom"))
            {
                await ctx.Response.WriteAsync("You've hit a custom middleware!");
            }
            else
            {
                await _next(ctx);
            }
        }
    }
}
```

Лістинг LoggingMiddleware.cs:

```
using static System.Net.Mime.MediaTypeNames;

namespace MiddlewareSandbox.Middlewares
{
    public class LoggingMiddleware
    {
        private RequestDelegate _next;
        private ILogger<LoggingMiddleware> _logger;


        public LoggingMiddleware(RequestDelegate next, ILogger<LoggingMiddleware> logger)
        {
            _next = next;
            _logger = logger;
        }

        public async Task Invoke(HttpContext ctx)
        {
            _logger.LogInformation($"Request: {ctx.Request.Method} {ctx.Request.Path}");
            await _next(ctx);
        }
    }
}
```

Лістинг CheckApiKeyMiddleware.cs:

```
namespace MiddlewareSandbox.Middlewares
{
    public class CheckApiKeyMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly string _validApiKey = "my-secret-key";

        public CheckApiKeyMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task Invoke(HttpContext ctx)
        {
            if (!ctx.Request.Headers.TryGetValue("X-API-KEY", out var apiKey) || apiKey != _validApiKey)
            {
                ctx.Response.StatusCode = StatusCodes.Status403Forbidden;
                await ctx.Response.WriteAsync("Forbidden: Invalid API Key");
```

```
            return;
        }

        await _next(ctx);
    }
  }
}
```

Лістинг Program.cs:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using MiddlewareSandbox.Middlewares;
using System.Threading.Tasks;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();


app.UseMiddleware<LoggingMiddleware>();
app.UseMiddleware<CheckApiKeyMiddleware>();
app.UseMiddleware<CounterMiddleware>();
app.UseMiddleware<CustomMiddleware>();

app.Run();
```

Результат:

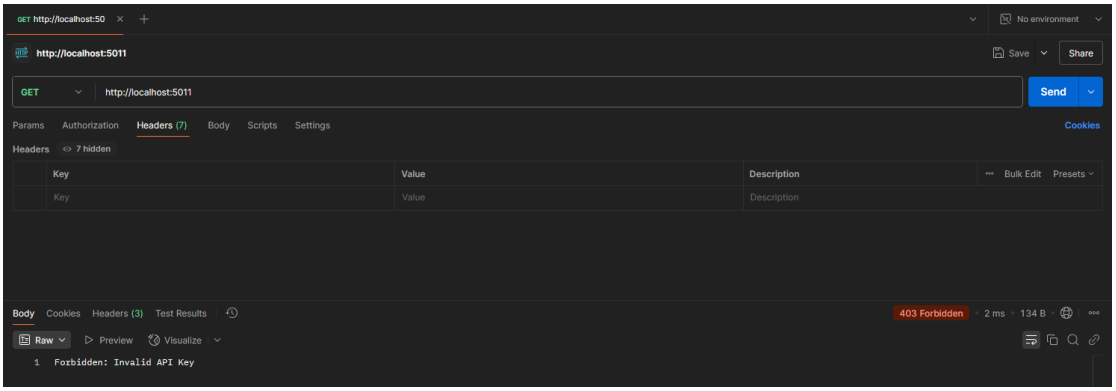

Рис. 1 Запит без наявного api-ключа

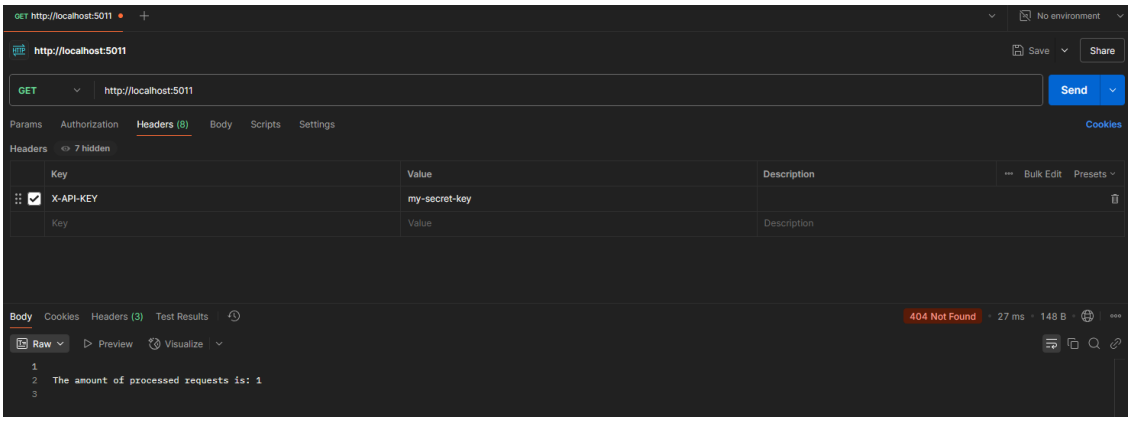

Рис. 2 Запит з наявним api-ключем

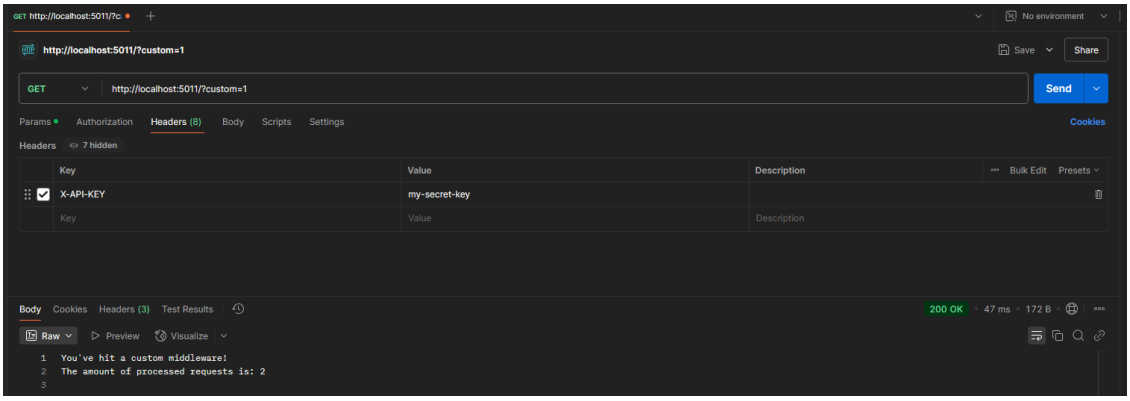

Рис. 3 Запит, що містить параметр “custom”

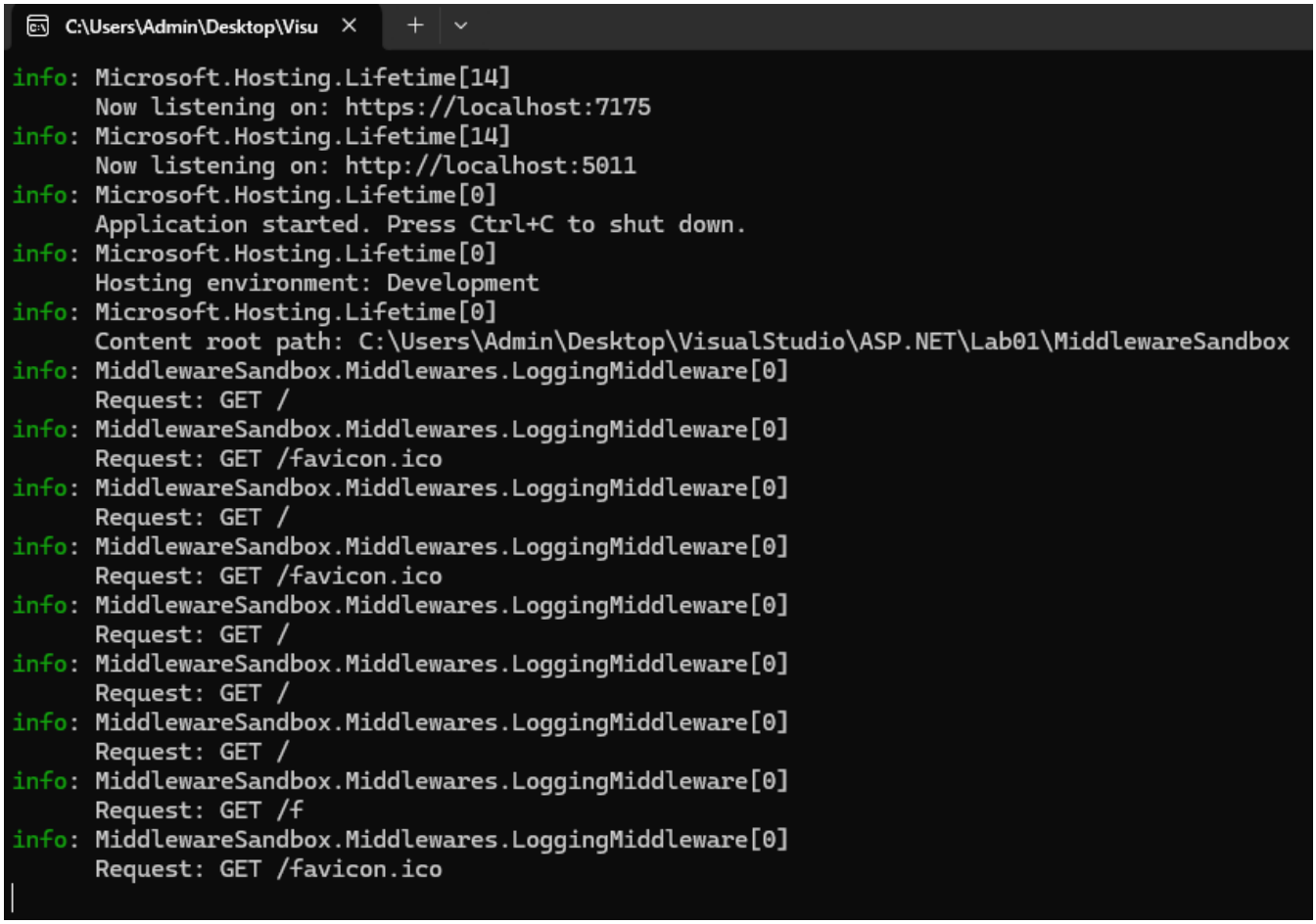

Рис. 4 Виведення запитів в консоль

Посилання на репозиторій: https://github.com/JanRizhenko/ASP.NET-course

Висновок: у результаті виконання цієї лабораторної роботи я ознайомитися з основними принципами роботи ASP.NET. Набув навичок обробки запитів з використанням middleware. Опанував роботу з .NET CLI.